KAM-WM: Kinematic Affordance Maps from Latent World Models for Robot Manipulation

Anonymous Author(s)
Affiliation
Address
email

Abstract: Learning manipulation from few demonstrations requires visual priors that capture not only *where* to interact, but also *how* the interaction should begin; static priors such as segmentation masks encode only the former. We present **KAM-WM**, a framework that extracts a coarse directional interaction cue from a frozen latent video world model without rollout or world-model fine-tuning. KAM-WM queries a Flow Matching image-to-video backbone once and interprets its single-step latent velocity as a *Kinematic Affordance Map* (KAM), which provides task-conditioned interaction regions and coarse motion structure. A lightweight Perceiver compresses KAM into tokens that condition a diffusion policy together with RGB observations and proprioception. Across LIBERO and RoboTwin 2.0, KAM-WM reaches 90.6% average success on LIBERO and achieves 65.7% and 22.4% success rates in the Easy and Hard settings on RoboTwin 2.0, respectively. Controlled comparisons against a zero-order mask prior suggest that part of the gains comes from directional information beyond spatial localization alone. These results indicate that, in the evaluated settings, a frozen video model can provide a useful first-order visual prior for control without the test-time cost of future rollout.

Keywords: Robot Manipulation, Latent World Models, Imitation Learning

1 Introduction

Learning manipulation from few demonstrations requires visual priors that indicate not only *where* to attend, but also *how* the interaction should begin. Static priors such as segmentation masks help localization, yet remain zero-order cues: they mark relevant objects or regions without encoding approach direction. This is limiting for tasks such as hanging a mug on a hook, where location alone does not determine a successful motion. As illustrated in Figure 1(a), a bottle mask may cover the whole object, whereas a more useful prior would emphasize the lower graspable region together with the gripper's approach motion.

Pretrained video models are an appealing source of this directional information, since future-frame prediction requires encoding how objects move and interact. Most robotics methods that use this knowledge, however, roll out future frames as visual subgoals or co-train the backbone with actions. Both add cost: iterative denoising increases test-time latency, and fine-tuning a large video model is expensive in the low-data regime we target. This raises a natural question: can motion knowledge in a frozen video model be read off directly, without generating future frames?

Our starting point is a simple empirical observation: when a frozen text-conditioned Wan 2.2 image-to-video model is queried at the first denoising step, its high-response regions often align with task-relevant contact structure and the robot embodiment. In a bottle manipulation scene, for example, the response highlights the lower graspable region and approaching arm, suggesting both sharper localization and a coarse interaction cue.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

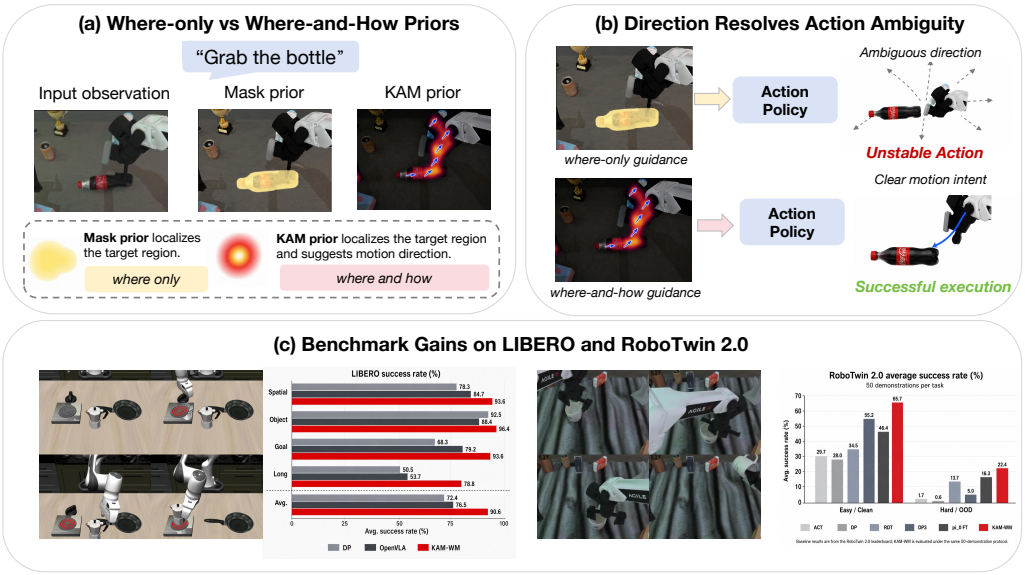

Figure 1: **KAM-WM provides where-and-how cues for low-data manipulation.** (a) KAM highlights interaction-relevant regions and motion cues beyond object masks. (b) These tokens condition the diffusion policy together with RGB and proprioception. (c) KAM-WM improves performance on LIBERO and RoboTwin 2.0.

This observation is consistent with the Flow Matching training objective [1, 2]. For a conditional video model, the single-step latent velocity at the high-noise endpoint (t=1.0) estimates a model-dependent displacement toward plausible future video latents conditioned on the current observation and language instruction; we use this field as a **Kinematic Affordance Map (KAM)**, whose magnitude highlights task-conditioned response regions and whose normalized latent response provides structure beyond a binary or soft mask. We do not treat this latent direction as a metrically calibrated 3D motion field, but as a demonstration-grounded visual prior that extends zero-order masks while avoiding multi-step rollout and world-model fine-tuning.

KAM-WM turns this intuition into a control interface. We compress the dense latent velocity field into 8 tokens with a lightweight Perceiver, and use them to condition a 1D U-Net Diffusion Policy together with multi-view RGB observations and proprioception. We do not interpret the latent field as a direct 3D action plan; KAM is a visual prior rather than an explicit geometric planner. The policy learns from demonstrations how to align this latent field with the robot action space.

We evaluate KAM-WM on LIBERO [3] and RoboTwin 2.0 [4]. On LIBERO, KAM-WM achieves 90.6% average success. On RoboTwin 2.0, under a 50-demonstration protocol, it reaches 65.7% Easy success across 50 tasks, exceeding the leaderboard ACT, π_0 FT, and DP3 Easy averages by 36.0, 19.3, and 10.5 points, respectively. Because these baselines are taken from the leaderboard, we treat the comparison as contextual rather than a controlled implementation comparison. Ablations with zero-order masks and KAM variants suggest that the gains are strongest when interaction direction matters and reflect information beyond localization alone.

The contributions of this work are:

- We propose extracting a single-step latent velocity field from a frozen Flow Matching image-to-video model as a first-order visual prior for low-data manipulation.
- We introduce a lightweight Perceiver-token interface that conditions a diffusion policy on this dense latent field without video rollout or world-model fine-tuning.
- We evaluate KAM-WM on LIBERO and RoboTwin 2.0, with ablations indicating that the gains are largest on tasks where directional interaction cues matter.

2 Related Work

Static visual and affordance representations. Visual representation learning has provided strong spatial and semantic features for robot learning through language supervision, masked reconstruction, segmentation, and robotics-oriented pre-training on human or embodied video [5, 6, 7, 8, 9, 10, 11, 12]. Affordance-based methods go further and predict where interaction can occur, using actionability scores, contact regions, or structured pick-and-place maps [13, 14, 15, 16, 17]. These representations are effective at localizing task-relevant objects, but they remain largely zero-order: they indicate *what* or *where* to attend to, while the approach direction and post-contact motion are left for the policy to infer. KAM-WM is complementary to this line: it reads a first-order velocity field from a frozen video model, adding a directional cue without requiring affordance labels.

Video world models and action-conditioned foundation models. Generative video and world models have been used for visual planning, subgoal-like guidance, and inverse dynamics [18, 19, 20]. In parallel, large Vision-Language-Action models learn action-conditioned representations from large-scale robot datasets [21, 22, 23, 24]. These models encode rich temporal and semantic priors, but adapting or using them for low-data manipulation often requires action fine-tuning, action co-training, or multi-step video rollout, which are costly and adds test-time latency. KAM-WM follows the opposite design point: the video backbone stays frozen and is queried once at the noise endpoint, so the temporal prior is reused without future-frame generation or any backbone update.

Motion and flow priors for policy learning. A related line of work conditions robot policies on predicted motion, such as object flow, point trajectories, or any-point tracking, to signal how the scene may change under an action [25, 26, 27, 28]. These methods show that motion prior can be valuable when static localization is insufficient, but they typically depend on a dedicated flow or tracking module, curated trajectory supervision, or an explicit procedure for converting 2D motion into robot actions. KAM-WM differs in where the motion signal comes from: rather than training a flow predictor, it extracts a latent velocity field from a general-purpose video model in a single query and uses it as a coarse visual prior. We therefore do not provide head-to-head comparisons with ATM, Track2Act, Im2Flow2Act, or GeneralFlow, whose flow or tracking modules are trained on different data sources and supervision.

3 The KAM-WM Framework

KAM-WM has two stages. First, it queries a frozen image-to-video model once at the start of an episode to obtain a task-conditioned Kinematic Affordance Map (KAM). Second, it compresses this dense field into a few tokens that condition a diffusion policy throughout closed-loop execution. The key idea is to use the model not for video rollout, but for a single-step latent velocity field whose magnitude highlights task-relevant regions and whose normalized response adds directional structure for the policy. Figure 2 illustrates the architecture.

3.1 Single-Step Latent Velocity Extraction

We read a task-conditioned motion cue from a frozen image-to-video model in a single forward pass, rather than by generating future frames. We use the official Wan2.2-TI2V-5B release [29] as the backbone, building on Rectified Flow [2, 1], and query it once at the maximum-noise endpoint. Let $\mathbf{x}_1 \sim \mathcal{N}(0, \mathbf{I})$ be a pure-noise latent and $\mathbf{x}_0 \sim q(\mathbf{x} \mid c)$ a clean future video latent conditioned on $c = [o_1, l]$, with initial observation o_1 and language instruction l. We use a denoising-time convention where t=1.0 is the noise endpoint and t=0.0 the data endpoint; Wan 2.2 is trained with target $\mathbf{v}_{\text{target}} = \mathbf{x}_1 - \mathbf{x}_0$. Under straight interpolation $\mathbf{x}_t = t\,\mathbf{x}_1 + (1-t)\,\mathbf{x}_0$, a model trained with squared error estimates

$$\mathbf{v}_\theta(\mathbf{x}_t, t, c) \approx \mathbb{E}[\mathbf{x}_1 - \mathbf{x}_0 \mid \mathbf{x}_t, c]\,. \tag{1}$$

At t=1.0, the input is $\mathbf{x}_t = \mathbf{x}_1$, giving

$$\mathbf{v}_\theta(\mathbf{x}_1, 1.0, c) \approx \mathbf{x}_1 - \mathbb{E}[\mathbf{x}_0 \mid c]\,. \tag{2}$$

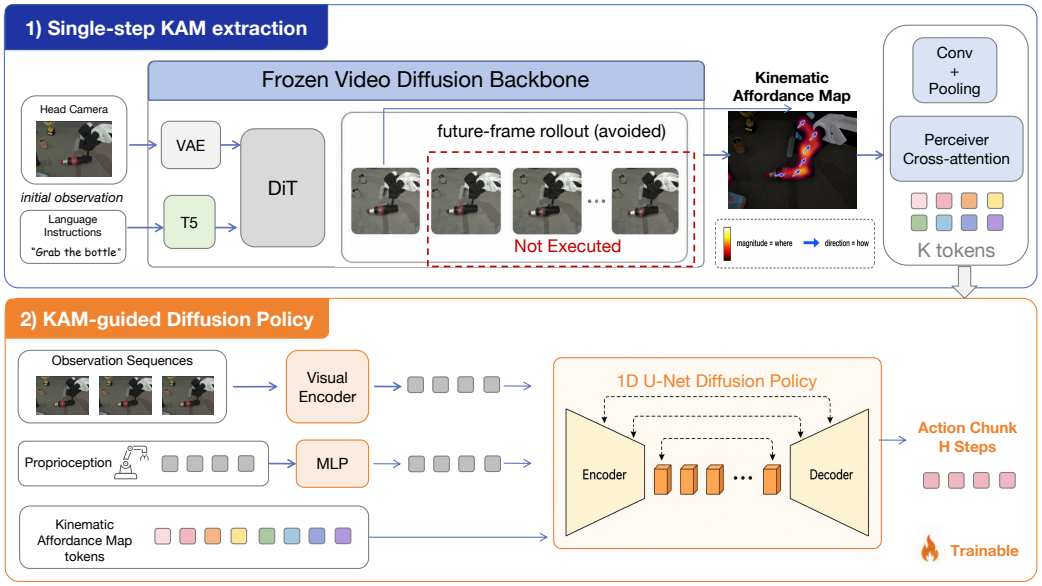

Figure 2: **KAM-WM framework.** Given the initial observation and language instruction, KAM-WM queries a frozen Flow Matching image-to-video backbone once at t=1.0, with no future-frame generation, and reads the resulting single-step latent velocity field as a Kinematic Affordance Map. A Perceiver compresses this dense field into K dynamic tokens, which condition a 1D U-Net Diffusion Policy together with current RGB observations and proprioception to predict H-step action chunks.

With $\mathbf{x}_1$ fixed, variation across (o_1, l) is dominated by the conditional term $\mathbb{E}[\mathbf{x}_0 \mid c]$, so Eq. 2 is a conditional latent estimate up to a constant offset. We use it as a visual prior rather than a calibrated motion field: a single noise-endpoint query depends on the observation and instruction, with no multi-step denoising.

3.2 Kinematic Affordance Maps

KAM-WM records the single-step latent velocity field

$$\mathbf{V}_{\text{prior}} = \mathbf{v}_\theta(\mathbf{x}_1, 1.0, [o_1, l]) \in \mathbb{R}^{C\times F\times H\times W}, \tag{3}$$

which we call a **Kinematic Affordance Map (KAM)**. The frame axis F is the model's parallel latent prediction over a fixed temporal window in one forward pass, with no autoregressive generation. KAM therefore remains in the video model's latent space: it is not decoded into future frames and is not interpreted as optical flow or a calibrated 3D displacement. For visualization and ablations, we separate a channel-axis magnitude from a normalized response:

$$\begin{aligned} A_{\text{kam}}(f,h,w) &= \|\mathbf{V}_{\text{prior}}(:,f,h,w)\|_2, \\ \widehat{\mathbf{V}}_{\text{prior}}(:,f,h,w) &= \frac{\mathbf{V}_{\text{prior}}(:,f,h,w)}{A_{\text{kam}}(f,h,w)+\epsilon}. \end{aligned} \tag{4}$$

The magnitude highlights where the model has a task-conditioned response, while the normalized response preserves coarse orientation structure in the latent field. These two views are useful for analysis, but unless noted, the policy consumes the dense field $\mathbf{V}_{\text{prior}}$ through the Perceiver so that localization and directional structure are provided jointly.

3.3 Dynamic Token Compression

The raw KAM tensor is too dense to use directly as a policy condition. We compress it with a Perceiver cross-attention bottleneck [30]: a $1\times1\times1$ projection and 3D adaptive average pooling produce a

token grid, which is flattened, augmented with learned positional embeddings, and compressed by K learnable queries,

$$\begin{aligned} \mathbf{Z} &= \mathrm{Flatten}(\mathrm{Pool}(\phi_{1\times1\times1}(\mathbf{V}_{\mathrm{prior}}))) + \mathbf{E}_{\mathrm{pos}}, \\ \mathbf{H}_{\mathrm{dyn}} &= \mathrm{Perceiver}(\mathbf{Q}, \mathbf{Z}) \in \mathbb{R}^{K\times d_k}, \end{aligned} \tag{5}$$

where $\mathbf{Q}$ are the learnable queries and $\mathbf{Z}$ the pooled token grid. This keeps the prior compact while preserving coarse spatial structure.

3.4 KAM-Conditioned Diffusion Policy

KAM-WM uses a 1D U-Net Diffusion Policy [31] as the action policy. The KAM prior is already language-conditioned through extraction; to make the RGB stream instruction-aware, we modulate the observation encoder Ψ with FiLM [32] using the frozen Wan 2.2 text embedding $\Phi(l)$. The policy conditions on multi-view RGB, proprioception, and the fixed KAM tokens through a global feature. At control step τ, with proprioceptive state s_τ and diffusion step k,

$$\begin{aligned} \mathbf{g}_\tau &= [\Psi(o_\tau, \Phi(l));\ \mathrm{vec}(\mathbf{H}_{\mathrm{dyn}});\ s_\tau], \\ \widetilde{\mathbf{a}}_k &= \alpha_k \mathbf{a}_{\tau:\tau+H_a} + \sigma_k \boldsymbol{\epsilon}, \\ \mathcal{L}_{\mathrm{DP}} &= \mathbb{E}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\widetilde{\mathbf{a}}_k, k, \mathbf{g}_\tau)\|_2^2\right]. \end{aligned} \tag{6}$$

KAM enters only as a conditioning input, never as an action target. By default it is extracted once per episode and held fixed, so the backbone runs a single query per episode while the policy stays closed-loop on RGB and proprioception.

4 Experiments

Our experiments address three questions. First, does a single-step latent velocity prior improve manipulation in the evaluated low-data settings? Second, are the observed gains consistent with a first-order directional cue rather than spatial localization alone? Third, what does this prior cost in inference latency and trainable parameters? We study the first two on LIBERO and RoboTwin 2.0, with controlled comparisons against a zero-order mask prior and an analysis of the extraction timestep, and report efficiency in Section 4.5.

4.1 Experimental Setup

Implementation. We use the frozen Wan2.2-TI2V-5B backbone [29], querying its video DiT once at the noise endpoint $t{=}1.0$ to read the single-step latent velocity field $\mathbf{V}_{\mathrm{prior}} \in \mathbb{R}^{48\times21\times50\times68}$ from the first head-camera observation; its text encoder supplies the FiLM embeddings used by the policy. The backbone is never updated, and we use neither future frames nor simulator state. Only three components are trained: a Perceiver bottleneck that compresses $\mathbf{V}_{\mathrm{prior}}$ into $K{=}8$ dynamic tokens, a per-view ResNet-18 encoder with text-modulated FiLM, and a 1D U-Net Diffusion Policy with action horizon $H_a{=}16$. KAM is extracted from the head camera, while the policy consumes both head and wrist-camera RGB together with proprioception (14-DoF dual-arm for RoboTwin 2.0, single-arm for LIBERO). Full optimizer, schedule, and augmentation settings are in Appendix A.

Benchmarks. **LIBERO** [3] comprises 40 tasks across four suites (Spatial, Object, Goal, Long); we follow the standard 50-demonstration protocol, train one multi-task policy per suite (500 demonstrations each), and report per-suite success rates. On LIBERO, DP and OpenVLA results are taken from the OpenVLA paper under the same benchmark setting. **RoboTwin 2.0** [4] contains 50 contact-rich bimanual tasks; we train each independently from 50 demonstrations and evaluate over 100 rollouts under *Easy* and *Hard* settings, the latter randomizing object positions, textures, and lighting. Baseline results are taken from the RoboTwin 2.0 leaderboard; KAM-WM and prior-type ablations are evaluated under the same 50-demonstration setting with 100 rollouts per task. We report representative task results together with the full 50-task average.

4.2 Results on LIBERO

KAM-WM improves LIBERO performance under the standard 50-demo setting. As shown in Table 1, KAM-WM reaches 90.6% average success, exceeding the DP and OpenVLA results reported in the OpenVLA paper by 18.2 and 14.1 points, respectively. The gains are most visible on the Goal and Long suites, where policies must maintain task intent over longer interactions and recover from intermediate state changes. On the Long suite, KAM-WM improves from 50.5% to 78.8% over DP and from 53.7% to 78.8% over OpenVLA. This pattern is consistent with KAM acting as a task-conditioned visual prior rather than only a static object localizer.

Table 1: **LIBERO success rates.** Results are reported in the standard 50-demonstration LIBERO setting. KAM-WM trains one multi-task policy per suite, and other results are taken from [24].

Method	Spat.	Obj.	Goal	Long	Avg.
DP [31]	78.3	92.5	68.3	50.5	72.4
OpenVLA [24]	84.7	88.4	79.2	53.7	76.5
KAM-WM	**93.6**	**96.4**	**93.6**	**78.8**	**90.6**

4.3 Results on RoboTwin 2.0

KAM-WM achieves the highest 50-task average success among the compared methods under the 50-demonstration protocol. Table 2 reports representative tasks together with the full 50-task average. KAM-WM reaches 65.7% Easy success across 50 tasks. Compared with leaderboard baselines under the same 50-demonstration setting, KAM-WM exceeds the reported DP3 Easy 50-task average of 55.2%. Because baseline rows are taken from the leaderboard rather than reproduced in our codebase, we treat these comparisons as contextual rather than controlled implementation comparisons. The largest displayed gains appear on tasks involving directional contact, object placement, or bimanual coordination, such as *Click Alarmclock*, *Hanging Mug*, *Place Cans Plasticbox*, and *Handover Block*. These tasks are consistent with the intended role of KAM as a single-step latent velocity field used by the policy.

Table 2: **RoboTwin 2.0 success rates.** Baseline results are taken from the RoboTwin 2.0 leaderboard; KAM-WM and prior-type ablations are evaluated under the same 50-demonstration setting with 100 rollouts per task. We report representative task results together with the full 50-task average.

Config	Method	Click Alarm.	Hang Mug	Cans Plastic.	Burger Fries	Open Laptop	H'over Mic	H'over Block	Press Stapler	Shake Bottle	Sel. Avg.	50-task Avg.
Easy	ACT [33]	32	7	16	49	56	85	42	31	74	43.6	29.7
	DP [31]	61	8	40	72	49	53	10	6	65	40.4	28.0
	RDT [34]	61	23	6	50	59	90	45	41	74	49.9	34.5
	DP3 [35]	77	17	48	72	82	**100**	70	69	**98**	70.3	55.2
	π_0 (FT) [23]	63	11	34	80	**85**	98	45	62	97	63.9	46.4
	KAM-WM	**96**	**47**	**87**	**96**	84	95	**85**	**71**	93	**83.8**	**65.7**
Hard	ACT [33]	4	0	0	0	0	0	0	6	10	2.2	1.7
	DP [31]	5	0	0	0	0	0	0	0	8	1.4	0.6
	RDT [34]	12	**16**	5	27	32	31	14	24	45	22.9	13.7
	DP3 [35]	14	1	3	18	7	3	0	3	19	7.6	5.0
	π_0 (FT) [23]	11	3	2	4	46	13	8	29	**60**	19.6	16.3
	KAM-WM	**45**	9	**25**	**40**	**69**	**80**	**35**	**31**	59	**43.7**	**22.4**

The Hard setting remains difficult for all methods. KAM-WM reaches 22.4% Hard success across 50 tasks, compared with leaderboard results of 16.3% for π_0, 1.7% for ACT, and 0.6% for DP; the leaderboard rows report 13.7% for RDT and 5.0% for DP3. The displayed tasks illustrate per-task behavior, while the 50-task average is the primary aggregate metric. KAM-WM tends to gain most on tasks with asymmetric approach geometry, where identifying the interaction side is important for success.

4.4 Ablation Studies

We run three diagnostic studies: 1) the conditioning prior (KAM vs. a zero-order mask), 2) the KAM extraction timestep, and 3) behavior in a lower-data regime. All three hold the policy architecture and protocol fixed and are diagnostic rather than full-benchmark sweeps; additional prior comparisons on RoboTwin tasks are in Appendix B.

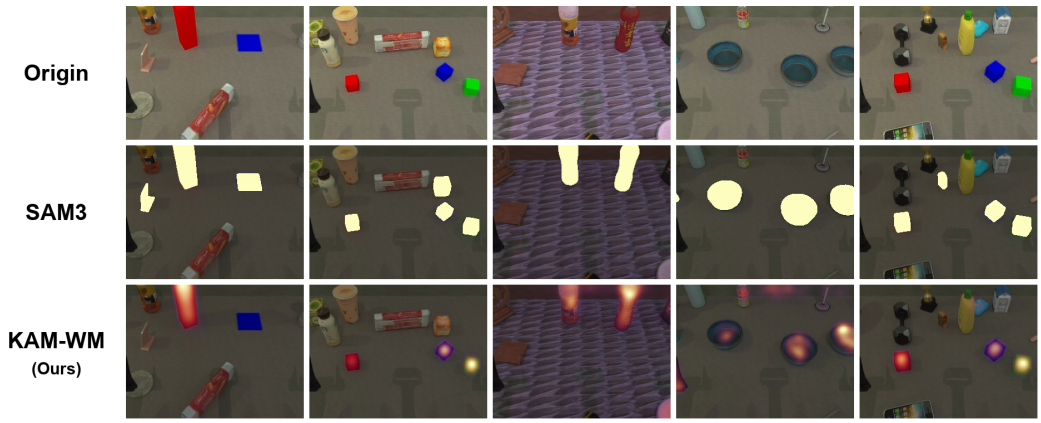

Figure 3: **Qualitative comparison of mask and KAM priors.** Mask priors localize target regions but remain zero-order spatial prior. KAM highlights more selective interaction regions and adds coarse orientation prior from the latent velocity field under the same policy interface.

Prior type. We compare KAM with a zero-order mask prior while keeping the policy architecture fixed. The mask baseline uses SAM 3 [36] masks conditioned on the task instruction, rendered at the KAM resolution and passed through the same Perceiver bottleneck, so only the conditioning signal changes from a language-conditioned mask to KAM. As shown in Table 3, KAM raises the Easy average from 67.8% to 77.0% and the Hard average from 24.0% to 28.6%, with the largest Easy gains on *Hanging Mug* and *Place Empty Cup*, where approach direction and post-contact motion matter. KAM does not always win: on *Click Bell* it improves Easy but drops from 34% to 18% on Hard, and a similar drop appears on *Place Shoe* (22% vs 10% Hard). A plausible explanation is that the latent velocity direction can be more sensitive to texture and lighting drift than mask-based localization when precise point contact is required. Figure 3 shows qualitative examples.

Extraction timestep. We study where along the Flow Matching trajectory to extract the field. Reading at any $t < 1.0$ requires first denoising from the noise endpoint down to the target step, restoring the multi-step cost we sought to avoid, so t=1.0 is the only strictly rollout-free operating point. Table 4 shows the best timestep is task dependent: t=0.0 or t=0.5 can help some tasks, and t=1.0 is not oracle-best. We adopt t=1.0 because it gives a competitive prior from one fixed query without any partial video generation.

Table 3: **Prior-type ablation on RoboTwin 2.0.** KAM is compared with a zero-order mask prior under the same policy interface.

Task	Easy		Hard	
	+Mask	**+KAM**	+Mask	**+KAM**
Adjust Bottle	96	**100**	76	**87**
Click Bell	83	**94**	**34**	18
Hanging Mug	24	**47**	**9**	**9**
Lift Pot	**90**	89	13	**17**
Place Can Basket	**42**	40	15	**26**
Place Cans Plasticbox	**88**	87	4	**25**
Place Empty Cup	49	**79**	4	**6**
Place Shoe	45	**64**	**22**	10
Shake Bottle	**93**	**93**	39	**59**
Average	67.8	**77.0**	24.0	**28.6**

Low-data regime. To probe whether first-order priors help as demonstrations grow scarcer, we compare the Diffusion Policy backbone, the same backbone with a zero-order mask prior, and DP+KAM-WM under 50-demo and 20-demo Easy settings, using the same checkpoint-selection rule for all methods (best of epochs {100, 300, 600}). The six tasks span direction-sensitive placement, point contact, geometry-sensitive lifting, repeated contact, and localization-dominant placement. Table 5 shows the mask prior is neutral at 50 demos and mixed at 20 demos on this subset, whereas 20-demo KAM is uniformly non-negative relative to the backbone and raises the subset average from 45.0% to 66.7%. We therefore view KAM as promising in lower-data regimes, while leaving full-suite 20-demo evaluation to future work.

Table 4: **Extraction-timestep ablation.** Later timesteps can improve some tasks, but t=1.0 is the only rollout-free setting.

Task	t=0.0		t=0.5		t=1.0	
	Easy	Hrd.	Easy	Hrd.	Easy	Hrd.
Beat Hammer	**91**	**17**	88	12	88	12
Cans Plasticbox	60	26	85	**27**	**87**	25
Hanging Mug	**59**	**11**	44	7	47	9
Handover Block	90	**42**	**91**	33	56	35
Average	75	**24**	**77**	20	70	20

Table 5: **Targeted low-data diagnostic.** Easy-setting success rates on six representative RoboTwin 2.0 tasks.

Task	50-demo			20-demo		
	DP	+Mask	+KAM	DP	+Mask	+KAM
Adjust Bottle	100	100	100	60	50	90
Lift Pot	90	90	89	35	45	55
Click Bell	95	95	94	40	30	75
Shake Bottle	90	85	93	75	70	75
Open Laptop	90	85	84	55	55	75
Place Empty Cup	25	35	79	5	10	30
Average	81.7	81.7	89.8	45.0	43.3	66.7

4.5 Efficiency Analysis

Inference efficiency. KAM-WM avoids multi-step video rollout during policy inference. With 10 denoising steps, the action-policy latency is **99 ms** per action chunk, compared with **380 ms** for our fine-tuned π_0 implementation on the same evaluation workstation. This per-chunk number excludes the one-time KAM extraction, which costs **895 ms** once per episode and is then amortized over the rollout. Perceiver compression adds less than **1 ms**, and cached KAM tensors of about **6.0 MB** per episode keep steady-state inference dominated by the policy.

Parameter efficiency. KAM-WM keeps the 5B video backbone frozen and precomputes the KAM tensors once per episode, so it is not in the policy-training loop. The trainable components are the per-view ResNet-18 encoder, the 93.4M 1D U-Net policy, and the 2.1M Perceiver bottleneck, totaling roughly 120M excluding frozen Wan parameters. The KAM-specific trainable module is the 2.1M Perceiver. KAM-WM reaches 65.7% Easy 50-task success, compared contextually with the 28.0% DP leaderboard average. Compared with fully fine-tuning the 3B π_0 model, KAM-WM trains roughly $25\times$ fewer parameters while reaching 65.7% versus 46.4% Easy success. The prior-type ablation in Table 3 further keeps the architecture fixed and swaps only the conditioning signal, reducing the chance that the gains are explained by added policy capacity alone.

5 Limitations, Discussion, and Conclusion

Limitations and Discussion. KAM-WM has five main limitations. First, it extracts KAM once per episode from the first head-camera frame, which keeps inference lightweight but lets the prior go stale under long horizons, major scene changes, or occlusion; refreshing it at task-critical moments is a natural extension. Second, the single-step velocity field is tied to the Flow Matching parameterization and should be re-examined for other video generators. Third, KAM is a coarse visual-motion prior rather than an explicit 3D plan, so precise contact or insertion may need tighter geometric, tactile, or multi-view feedback. Fourth, our evaluation is simulation-only: Wan2.2 rollouts from real robot images provide only a qualitative sanity check, not real-world validation. Fifth, we report single-run success rates rather than multi-seed confidence intervals, leaving training variance in the low-data regime to future work.

Conclusion. We introduced **KAM-WM**, which reuses the single-step latent velocity field of a frozen Flow Matching image-to-video backbone as a first-order visual prior for low-data robot manipulation. Rather than rolling out future frames or fine-tuning the backbone, KAM-WM reads a task-conditioned motion cue in a single query and lets a lightweight policy learn to use it. Across LIBERO and RoboTwin 2.0 it improves over reported DP/OpenVLA and leaderboard baselines in contextual comparisons, while training over $25\times$ fewer parameters than the fine-tuned π_0 comparison. More broadly, our results suggest that some directional structure encoded by a frozen video model is accessible without test-time future generation, making such models a useful source of first-order visual priors in the evaluated manipulation settings.

References

[1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.02747.

[2] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[3] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 36:44776–44791, 2023.

[4] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021.

[6] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.

[7] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Soyer, V. Kinnunen, H. Touvron, et al. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[8] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, 2023.

[9] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning (CoRL)*, pages 892–909. PMLR, 2022.

[10] I. Radosavovic, T. Xiao, S. James, P. Abbeel, J. Malik, and T. Darrell. Real-world robot learning with masked visual pre-training. In *Conference on Robot Learning (CoRL)*, 2022.

[11] A. Majumdar, K. Yadav, S. Lin, S. Peralta, et al. Where are we in the search for an artificial visual cortex for embodied intelligence? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[12] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.

[13] K. Mo, L. J. Guibas, M. Mukadam, A. Gupta, and S. Tulsiani. Where2Act: From pixels to actions for articulated 3d objects. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6813–6823, 2021.

[14] R. Wu, Y. Zhao, K. Mo, Z. Guo, Y. Wang, T. Wu, Q. Fan, X. Chen, L. Guibas, and H. Dong. VAT-Mart: Learning visual action trajectory proposals for manipulating 3d articulated objects. In *International Conference on Learning Representations (ICLR)*, 2022.

[15] S. Bahl, R. Mendonca, L. Chen, U. Jain, and D. Pathak. Affordances from human videos as a versatile representation for robotics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13778–13790, 2023.

[16] A. Zeng, P. Florence, J. Tompson, J. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee. Transporter networks: Rearranging the visual world for robotic manipulation. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 155 of *Proceedings of Machine Learning Research*, pages 726–747. PMLR, 2021.

[17] M. Shridhar, L. Manuelli, and D. Fox. CLIPort: What and where pathways for robotic manipulation. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 894–906. PMLR, 2022.

[18] Y. Du, M. Yang, P. Florence, and A. Zeng. UniPi: Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.

[20] K. Black, M. Nakamoto, P. Atreya, H. Walke, C. Finn, A. Kumar, and S. Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations (ICLR)*, 2024.

[21] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[22] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[23] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[24] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2025.

[25] C. Wen, X. Lin, J. I. R. So, K. Chen, Q. Dou, Y. Gao, and P. Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024. doi:10.15607/RSS.2024.XX.092. arXiv:2401.00025.

[26] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani. Track2act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2405.01527.

[27] M. Xu, Z. Xu, Y. Xu, C. Chi, G. Wetzstein, M. Veloso, and S. Song. Flow as the cross-domain manipulation interface. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2475–2499. PMLR, 2025. arXiv:2407.15208.

[28] C. Yuan, C. Wen, T. Zhang, and Y. Gao. General flow as foundation affordance for scalable robot learning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1541–1566. PMLR, 2025. arXiv:2401.11439.

[29] Wan Team, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[30] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, pages 4651–4664. PMLR, 2021.

[31] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[32] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[33] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[34] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: A diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[35] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Robotics: Science and Systems (RSS)*, 2024.

[36] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, et al. SAM 3: Segment anything with concepts. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2511.16719.

Appendix Overview

This appendix provides supplementary materials for the main paper, including:

- Appendix A: Detailed implementation and training parameters.
- Appendix B: Additional comparisons between KAM and zero-order mask priors.
- Appendix C: Complete 50-task RoboTwin 2.0 evaluation results.

A Implementation Details

Latent world model and prior extraction. We use Wan 2.2 5B [29] as a frozen image-to-video backbone. Given the first head-camera observation of an episode $o_1 \in \mathbb{R}^{3\times480\times640}$ and the language instruction l, we query the model once at the high-noise endpoint t=1.0. We record the resulting latent velocity field

$$\mathbf{V}_{\text{prior}} \in \mathbb{R}^{48\times21\times50\times68}$$

after a single forward pass through the DiT stack. No iterative denoising or future-frame generation is performed. The world-model backbone remains frozen during both training and evaluation.

KAM-WM Perceiver bottleneck. The Perceiver bottleneck compresses the dense KAM tensor into a small set of dynamic tokens. A $1\times1\times1$ Conv3D projection maps the channel dimension from 48 to 256, followed by GELU activation and 3D adaptive average pooling to $(7, 10, 17)$. This yields $M' = 1{,}190$ tokens. We add a learned positional embedding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1190\times256}$ initialized with a truncated normal distribution ($\sigma = 0.02$). $K = 8$ learnable latent queries attend to these tokens through one pre-norm multi-head cross-attention layer with 8 heads and hidden size 256, followed by a pre-norm feed-forward network with hidden size 4×256. The bottleneck contains approximately 2.1M trainable parameters.

Visual encoder with text-modulated FiLM. The multi-view visual encoder Ψ is a ResNet-18 instantiated independently for each camera view. After each residual block, feature maps are modulated by FiLM layers [32]. The FiLM scale and shift parameters are predicted from the frozen Wan 2.2 text embedding $\Phi(l)$ using a small MLP. The ResNet and FiLM heads are trained jointly with the Perceiver and diffusion policy, while the Wan text encoder remains frozen.

1D U-Net Diffusion Policy. The downstream policy follows a standard 1D U-Net Diffusion Policy architecture [31]. It uses four downsampling stages with channel widths 256–512–1024–1024. The global condition concatenates language-modulated visual features, flattened KAM tokens ($K \times d_k = 2{,}048$ dimensions), and proprioceptive features. For RoboTwin 2.0, proprioception includes the 14-DoF dual-arm state and end-effector poses. For LIBERO, it uses the corresponding single-arm state. The action chunk horizon is $H_a = 16$. We use a DDPM scheduler with 100 denoising steps during training and 10 steps during inference.

Training configuration. For RoboTwin 2.0, each task is trained independently. For LIBERO, we train a single multi-task policy for each suite. All policies are trained for 3,000 gradient steps with batch size 64 on a single high-end GPU. The Perceiver, visual encoder, FiLM heads, and U-Net policy are optimized with AdamW using learning rate 1×10^{-4}, weight decay 10^{-4}, and cosine learning-rate decay. Training each model takes approximately 2–3 GPU hours. We apply random color jitter and horizontal flips to each camera view.

Prior caching during training. For the default setting used in the paper, KAM is extracted from the first observation of each demonstration episode and reused for all training windows from that episode. We therefore precompute and cache the KAM tensor once per episode. During training, the frozen 5B image-to-video backbone is removed from the hot path; the training loop reads cached priors and only updates the Perceiver, visual encoder, and diffusion policy.

Evaluation protocol. Each RoboTwin 2.0 task is evaluated over 100 rollouts. Unless otherwise stated, the reported results are from a single training run per method. The Easy setting follows RoboTwin's Clean setting, with a clean background and no distractors, while the Hard setting follows RoboTwin's Random setting, with complex backgrounds, challenging lighting, and 10 distractors in each scene. At test time, KAM is extracted once from the first observation of an episode and held fixed during closed-loop execution. The reported per-chunk policy latency excludes this one-time prior extraction, which takes approximately 895 ms on the evaluation workstation GPU. The policy latency is measured during closed-loop action prediction with 10 denoising steps and excludes simulator stepping, rendering, and environment reset time.

B Additional Comparison with Zero-Order Masks

Table 6: Additional comparison between zero-order mask priors and KAM priors. Success rates (%) are evaluated over 100 rollouts per task.

Task	**Easy**					**Hard**				
	DP	ACT	π_0	Mask	**KAM-WM**	DP	ACT	π_0	Mask	**KAM-WM**
Adjust Bottle	97	97	90	96	**100**	66	23	56	76	**87**
Click Bell	54	58	44	83	**94**	4	3	3	**34**	18
Hanging Mug	8	7	11	24	**47**	6	0	3	**9**	**9**
Lift Pot	39	88	84	**90**	89	9	0	**36**	13	17
Place Can Basket	18	1	**41**	**42**	40	8	0	5	15	**26**
Place Cans Plasticbox	40	16	34	**88**	87	4	0	2	4	**25**
Place Empty Cup	37	61	37	49	**79**	2	0	**11**	4	6
Place Shoe	23	5	28	45	**64**	7	0	6	**22**	10
Shake Bottle	65	74	**97**	93	93	28	10	**60**	39	59
Average	42.3	45.2	51.8	67.8	**77.0**	14.9	4.0	20.7	24.0	**28.6**

C Full Evaluation Results on RoboTwin 2.0

Table 7 reports the complete per-task evaluation on all 50 RoboTwin 2.0 tasks under Easy and Hard settings.

The per-task results indicate that KAM-WM performs especially well on tasks where localized contact and approach direction are important, including *Click Alarmclock*, *Hanging Mug*, *Place Burger Fries*, and *Handover Block*. These results are consistent with the main paper's hypothesis that the latent velocity field provides useful information beyond object-level localization.

KAM-WM is less consistently advantageous on tasks where the interaction is kinematically simple or where success depends more on semantic disambiguation and precise execution, such as *Shake Bottle* or *Grab Roller*. This pattern supports the interpretation of KAM as a coarse first-order visual prior rather than a complete action representation.

Table 7: **Complete RoboTwin 2.0 results over 50 tasks under Easy and Hard settings.** Success rates (%) are computed over 100 rollouts per task. Baseline results are taken from the RoboTwin 2.0 leaderboard; KAM-WM is evaluated under the same 50-demonstration setting.

Task	**Easy**						**Hard**					
	ACT	**DP**	**RDT**	**DP3**	π_0	**KAM**	**ACT**	**DP**	**RDT**	**DP3**	π_0	**KAM**
Adjust Bottle	97.0	97.0	81.0	99.0	90.0	**100.0**	23.0	0.0	75.0	3.0	56.0	**87.0**
Beat Block Hammer	56.0	42.0	77.0	72.0	43.0	**88.0**	3.0	0.0	**37.0**	8.0	21.0	12.0
Blocks Ranking RGB	1.0	0.0	3.0	3.0	19.0	**28.0**	0.0	0.0	0.0	0.0	**5.0**	3.0
Blocks Ranking Size	0.0	1.0	0.0	2.0	7.0	**20.0**	0.0	0.0	0.0	0.0	**1.0**	**1.0**

Continued on next page

Table 7 continued

Task	Easy						Hard					
	ACT	**DP**	**RDT**	**DP3**	π_0	**KAM**	**ACT**	**DP**	**RDT**	**DP3**	π_0	**KAM**
Click Alarmclock	32.0	61.0	61.0	77.0	63.0	**96.0**	4.0	5.0	12.0	14.0	11.0	**45.0**
Click Bell	58.0	54.0	80.0	90.0	44.0	**94.0**	3.0	0.0	9.0	0.0	3.0	**18.0**
Dump Bin Bigbin	68.0	49.0	64.0	**85.0**	83.0	72.0	1.0	0.0	32.0	**53.0**	24.0	47.0
Grab Roller	94.0	**98.0**	74.0	**98.0**	96.0	97.0	25.0	0.0	43.0	2.0	**80.0**	**80.0**
Handover Block	42.0	10.0	45.0	70.0	45.0	**85.0**	0.0	0.0	14.0	0.0	8.0	**35.0**
Handover Mic	85.0	53.0	90.0	**100.0**	98.0	95.0	0.0	0.0	31.0	3.0	13.0	**80.0**
Hanging Mug	7.0	8.0	23.0	17.0	11.0	**47.0**	0.0	0.0	**16.0**	1.0	3.0	9.0
Lift Pot	88.0	39.0	72.0	**97.0**	84.0	89.0	0.0	0.0	9.0	0.0	**36.0**	17.0
Move Can Pot	22.0	39.0	25.0	70.0	58.0	**87.0**	4.0	0.0	12.0	6.0	21.0	**28.0**
Move Pillbottle Pad	0.0	1.0	8.0	41.0	21.0	**47.0**	0.0	0.0	0.0	0.0	1.0	**10.0**
Move Playingcard Away	36.0	47.0	43.0	68.0	53.0	**82.0**	0.0	0.0	11.0	3.0	**22.0**	21.0
Move Stapler Pad	0.0	1.0	2.0	12.0	0.0	**39.0**	0.0	0.0	0.0	0.0	**2.0**	0.0
Open Laptop	56.0	49.0	59.0	82.0	**85.0**	84.0	0.0	0.0	32.0	7.0	46.0	**69.0**
Open Microwave	86.0	5.0	37.0	61.0	80.0	**96.0**	0.0	0.0	20.0	22.0	**50.0**	**50.0**
Pick Diverse Bottles	7.0	6.0	2.0	52.0	27.0	**57.0**	0.0	0.0	0.0	1.0	**6.0**	5.0
Pick Dual Bottles	31.0	24.0	42.0	60.0	57.0	**91.0**	0.0	0.0	13.0	1.0	12.0	**18.0**
Place A2B Left	1.0	2.0	3.0	**46.0**	31.0	31.0	0.0	0.0	1.0	2.0	1.0	**5.0**
Place A2B Right	0.0	13.0	1.0	**49.0**	27.0	21.0	0.0	0.0	1.0	0.0	**6.0**	4.0
Place Bread Basket	6.0	14.0	10.0	26.0	17.0	**59.0**	0.0	0.0	2.0	1.0	4.0	**14.0**
Place Bread Skillet	7.0	11.0	5.0	19.0	23.0	**60.0**	0.0	0.0	1.0	0.0	1.0	**8.0**
Place Burger Fries	49.0	72.0	50.0	72.0	80.0	**96.0**	0.0	0.0	27.0	18.0	4.0	**40.0**
Place Can Basket	1.0	18.0	19.0	**67.0**	41.0	40.0	0.0	0.0	6.0	2.0	5.0	**26.0**
Place Cans Plasticbox	16.0	40.0	6.0	48.0	34.0	**87.0**	0.0	0.0	5.0	3.0	2.0	**25.0**
Place Container Plate	72.0	41.0	78.0	86.0	88.0	**92.0**	1.0	0.0	17.0	1.0	**45.0**	39.0
Place Dual Shoes	9.0	8.0	4.0	13.0	15.0	**44.0**	0.0	0.0	**4.0**	0.0	0.0	**4.0**
Place Empty Cup	61.0	37.0	56.0	65.0	37.0	**79.0**	0.0	0.0	7.0	1.0	**11.0**	6.0
Place Fan	1.0	3.0	12.0	36.0	20.0	**42.0**	0.0	0.0	2.0	1.0	**10.0**	8.0
Place Mouse Pad	0.0	0.0	1.0	4.0	7.0	**49.0**	0.0	0.0	0.0	**1.0**	**1.0**	**1.0**
Place Object Basket	15.0	15.0	33.0	**65.0**	16.0	55.0	0.0	0.0	**17.0**	0.0	2.0	4.0
Place Object Scale	0.0	1.0	1.0	15.0	10.0	**30.0**	0.0	0.0	0.0	0.0	0.0	**1.0**
Place Object Stand	1.0	22.0	15.0	60.0	36.0	**73.0**	0.0	0.0	5.0	0.0	11.0	**20.0**
Place Phone Stand	2.0	13.0	15.0	44.0	35.0	**68.0**	0.0	0.0	6.0	2.0	7.0	**15.0**
Place Shoe	5.0	23.0	35.0	58.0	28.0	**64.0**	0.0	0.0	7.0	2.0	6.0	**10.0**
Press Stapler	31.0	6.0	41.0	69.0	62.0	**71.0**	6.0	0.0	24.0	3.0	29.0	**31.0**
Put Bottles Dustbin	27.0	22.0	21.0	**60.0**	54.0	50.0	1.0	0.0	4.0	**21.0**	13.0	10.0
Put Object Cabinet	15.0	42.0	33.0	**72.0**	68.0	50.0	0.0	0.0	**18.0**	1.0	**18.0**	10.0
Rotate QRcode	1.0	13.0	50.0	**74.0**	68.0	43.0	0.0	0.0	5.0	1.0	**15.0**	7.0
Scan Object	2.0	9.0	4.0	31.0	18.0	**48.0**	0.0	0.0	1.0	1.0	1.0	**2.0**
Shake Bottle	74.0	65.0	74.0	**98.0**	97.0	93.0	10.0	8.0	45.0	19.0	**60.0**	59.0
Shake Bottle Horizontally	63.0	59.0	84.0	**100.0**	99.0	92.0	4.0	18.0	51.0	25.0	51.0	**55.0**
Stack Blocks Three	0.0	0.0	2.0	1.0	17.0	**44.0**	0.0	0.0	0.0	0.0	0.0	**8.0**
Stack Blocks Two	25.0	7.0	21.0	24.0	42.0	**55.0**	0.0	0.0	2.0	0.0	1.0	**13.0**
Stack Bowls Three	48.0	63.0	51.0	57.0	66.0	**67.0**	0.0	0.0	17.0	5.0	24.0	**25.0**
Stack Bowls Two	82.0	61.0	76.0	83.0	**91.0**	89.0	0.0	0.0	30.0	6.0	**41.0**	25.0
Stamp Seal	2.0	2.0	1.0	18.0	3.0	**44.0**	0.0	0.0	0.0	0.0	**4.0**	2.0
Turn Switch	5.0	36.0	35.0	46.0	27.0	**56.0**	2.0	1.0	15.0	8.0	**23.0**	10.0
Average	29.7	28.0	34.5	55.2	46.4	**65.7**	1.7	0.6	13.7	5.0	16.3	**22.4**